SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA1707

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I N. Sree lakshmi(192511251)(Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: SREE

Annexure A – Scenario Based Assignment

Artificial Intelligence

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a

flood-affected region. The environment is highly dynamic—roads may suddenly become

inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic

estimate (straight-line distance), Partial real-time updates about blocked paths and Variable

movement costs depending on terrain and weather. However, due to urgency, the drone must

quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its

strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and

heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-

time constraints, optimality, and safety

1. AI-Powered Drones for Emergency Medicine Delivery

Introduction

A government uses AI-powered drones to deliver emergency medicines in flood-affected regions. The environment changes frequently due to blocked roads, bad weather, and varying terrain costs. The drone must find the safest and fastest route in real time.

i. Greedy Best-First Search

Greedy Best-First Search selects the path with the smallest heuristic value (estimated distance to the goal). It ignores the cost already travelled and focuses only on reaching the destination quickly.

Strengths:

- Fast decision-making.
- Requires less computation.
- Suitable for urgent situations.

Weaknesses:

- May choose unsafe or costly paths.
- Does not guarantee the shortest route.
- Performance decreases when the heuristic is inaccurate.

*ii. A Search**

A* Search uses the evaluation function:

f(n) = g(n) + h(n)

Where:

- g(n) = Actual cost from the start.
- h(n) = Estimated cost to the goal.

It balances travelled cost and estimated distance to find an optimal and safe route.

iii. Impact of Heuristic Accuracy

- Accurate heuristics help both algorithms reach the goal quickly.
- Greedy Search heavily depends on heuristic accuracy.
- A* is more reliable because it also considers the actual path cost.
- Poor heuristics increase search time and may produce inefficient paths.

iv. Effect of Dynamic Changes

Blocked roads and weather changes require the search algorithm to update its path.

- Greedy Search may repeatedly choose blocked paths.
- A* recalculates routes by considering updated movement costs, making it more reliable.

v. Recommended Algorithm

*A Search** is the best choice because it:

- Produces optimal paths.
- Considers both travel cost and estimated distance.
- Adapts better to changing environments.
- Improves safety and delivery reliability.

Conclusion: A* Search provides the best balance between speed, safety, and optimality for emergency drone navigation.

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across

hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption

and Traffic congestion. The system must continuously adjust signals based on traffic flow.

However the search space is extremely large, traffic patterns change dynamically and the

system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Smart City Traffic Signal Optimization

Introduction

A smart city uses AI to optimize traffic signals to reduce waiting time, fuel consumption, and congestion.

Problem Formulation

This is an optimization problem where the objective is to minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Traditional search methods are inefficient because:

- The search space is extremely large.
- Traffic conditions change continuously.
- Exploring every solution is computationally expensive.

i. Hill Climbing

Hill Climbing starts with an initial solution and continuously moves toward better neighboring solutions.

Advantages

- Simple implementation.
- Fast execution.
- Suitable for local improvements.

Limitations

- Gets stuck in local maxima.

- Cannot escape flat regions (plateaus).
- May fail to reach the global optimum.

ii. Local Maxima, Plateaus and Ridges

- Local Maximum: Traffic improves locally but is not globally optimal.
- Plateau: Many signal timings produce similar performance.
- Ridge: Small improvements require multiple coordinated changes, making progress difficult.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

- Occasionally accepts worse solutions.
- Escapes local maxima.
- Finds better global solutions.

Genetic Algorithm

- Uses selection, crossover, and mutation.
- Searches many solutions simultaneously.
- Produces high-quality solutions for large optimization problems.

iv. Recommended Approach

The Genetic Algorithm is recommended because it:

- Handles very large search spaces.
- Adapts to changing traffic conditions.
- Produces near-optimal solutions.
- Scales efficiently for smart cities.

Conclusion: Genetic Algorithms provide better scalability, adaptability, and optimization than traditional search techniques.

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

Mars Rover Navigation

Introduction

An autonomous Mars rover explores unknown terrain while avoiding hazards and collecting scientific samples.

i. Need for an Online Search Agent

An online search agent is required because:

- The rover has no complete map.
- It discovers obstacles while moving.
- Decisions must be made in real time.

Offline search cannot be used because the entire environment is unknown.

ii. Challenges

- Partial observability due to limited sensors.
- Dynamic environment with unexpected hazards.
- Limited energy and communication.
- Safety-critical navigation.

iii. Internal Model

The rover:

- Collects sensor information.
- Updates its internal map.
- Identifies obstacles.
- Replans routes whenever new hazards are detected.

iv. Comparison

Uninformed Search

- No heuristic guidance.
- Explores many unnecessary paths.

Informed Search

- Uses heuristics.
- Faster than uninformed search.

Online Search

- Learns while exploring.
- Continuously updates decisions.
- Best suited for unknown environments.

v. Recommended Approach

An Online Search Agent is the best choice because it:

- Handles uncertainty.
- Adapts to new information.
- Improves safety.
- Supports autonomous exploration.

Conclusion: Online search enables intelligent and safe exploration in unknown environments.

4. Scenario: A university is designing an AI system to automatically generate exam

timetables. The system must satisfy multiple constraints: No student should have overlapping

exams, Limited number of exam halls, Certain subjects must be scheduled before others,

Faculty availability constraints, Special accommodations for some students, The complexity

increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

University Exam Timetable Generation

Introduction

A university needs an AI system to generate exam schedules while satisfying multiple constraints.

Problem Formulation as CSP

Variables

- Exam subjects
- Time slots
- Exam halls

Domains

- Available dates
- Available time slots
- Available rooms

Constraints

- No overlapping exams for students.
- Limited hall availability.
- Subject order constraints.
- Faculty availability.
- Special accommodations.

ii. Backtracking Search

Backtracking:

- Assigns values one by one.
- Checks constraints.
- Returns to the previous assignment if a conflict occurs.

iii. Constraint Propagation (Forward Checking)

Forward checking:

- Removes invalid future choices.
- Detects conflicts early.

- Reduces unnecessary search.
- Improves efficiency.

iv. Real-World Challenges

- Thousands of students.
- Hundreds of courses.
- Changing faculty schedules.
- Last-minute updates.

Best Strategy
Combine Backtracking with Constraint Propagation because it improves scalability and reduces computation.

Conclusion: CSP with Forward Checking provides efficient and practical timetable generation.

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta

Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The

AI must complete against human players, Make decisions within strict time limits, Evaluate

complex game states with multiple possible moves, The decision tree is extremely large,

making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Strategic Game AI (Minimax & Alpha-Beta Pruning)

Introduction

A gaming company develops an AI opponent that must make intelligent decisions within strict time limits.

i. Minimax Algorithm

Minimax:

- Maximizes the AI's score.
- Minimizes the opponent's advantage.
- Evaluates future game states before selecting a move.

ii. Computational Challenges

- Very large game tree.
- High branching factor.
- Long computation time.
- Large memory requirements.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning:

- Eliminates branches that cannot affect the final decision.
- Reduces the number of evaluated nodes.
- Produces the same optimal result as Minimax with less computation.

iv. Evaluation Function

The evaluation function may consider:

- Number of remaining units.
- Health of units.
- Resource availability.
- Map control.
- Strategic position.
- Distance to objectives.

These factors estimate the strength of a game state.

v. Recommended Strategy

Use Minimax with Alpha-Beta Pruning because it:

- Maintains optimal decisions.
- Reduces search time.
- Meets real-time requirements.

- Performs efficiently in large game trees.

Conclusion: Alpha-Beta Pruning significantly improves Minimax performance while maintaining decision quality.

Overall Conclusion

Artificial Intelligence techniques such as A* Search, Hill Climbing, Genetic Algorithms, Online Search Agents, Constraint Satisfaction Problems, Minimax, and Alpha-Beta Pruning solve different real-world problems effectively. Selecting the appropriate algorithm based on the environment, constraints, uncertainty, and computational requirements leads to efficient, reliable, and intelligent decision-making systems.